

TRACE: Trajectory-Guided Attack Material Synthesis for Tool-Using Agents

Huang Yingkun¹ Zhuang Xiaoru²

¹China Electronics Data Corporation

²College of Mechanical and Electrical Engineering,
Shenzhen Polytechnic University

Abstract

Tool-using language-model agents introduce security risks that cannot be evaluated from final textual responses alone. In realistic agent settings, security outcomes depend on the full execution trajectory: which tools were invoked, how parameters were populated, whether authorization evidence was accepted, whether environment state changed, and whether the action was verified through readback. We present TRACE (Trajectory-guided Risk-Aware Candidate Evolution), the method underlying our GreedyAI submission to the OpenClaw Agent Security Attack Competition. TRACE formulates attack-material synthesis as constrained trajectory optimization: materials must pass text-level safety filters while still inducing target-specific tool-use trajectories in the official runtime. The framework combines target-local candidate families, a conservative Stage 1/2 risk gate, fresh Docker execution with the required Kimi-K2.5 base model, an audit-log trajectory oracle, greedy target-wise selection, and failure-driven mutation. In final-round development feedback, an early safe-boundary variant achieved 7/19 target success with an online score of 0.368, while the final TRACE variant achieved 15/19 target success with an online score of 0.789. These results are competition feedback rather than a universal benchmark, but they show that trajectory-aware synthesis can improve attack effectiveness under strict text-level safety constraints.

1 Introduction

Large language model agents are increasingly deployed with tools such as email clients, browsers, file systems, calendars, enterprise-office systems, e-commerce services, and smart-home controllers. Unlike ordinary chatbots, these agents observe external records, select tools, populate structured parameters, call services, and update environment states. This shift expands the security problem from content safety to behavioral safety.

For tool-using agents, inspecting only the final assistant response is insufficient. An agent may verbally refuse a risky request while still invoking a sensitive tool, or it may claim completion without changing the environment. A security evaluation must therefore examine tool invocation, parameter population, authorization evidence, state change, and readback.

The OpenClaw Agent Security Attack Competition captures this challenge in a controlled evaluation environment. Participants construct task instructions, optional skill files, and attachment records. These materials first pass text-level safety checks, including a skill/attachment scan and a task-instruction safety evaluation. Only then are execution traces

scored for target-specific tool-use effects. The central question is: how can a participant synthesize competition-compliant materials that pass text-level safety filters while still inducing target-specific tool-use trajectories?

We answer this question with TRACE, a trajectory-guided candidate search framework. It treats each target as a local constrained search problem, generates structurally different candidate families, filters unsafe variants before execution, runs accepted candidates in fresh OpenClaw Docker containers with Kimi-K2.5, scores audit logs, freezes successful targets, and mutates only unresolved targets.

Our contributions are threefold. First, we formulate OpenClaw attack-material synthesis as a constrained trajectory optimization problem, where text-level safety compliance and target-level tool-use success must be optimized jointly. Second, we propose TRACE, a trajectory-guided and risk-aware candidate evolution framework that combines target-local material generation, pre-execution safety filtering, audit-log feedback, and failure-driven mutation. Third, we provide an empirical study over 19 OpenClaw final-round targets, showing that text-level compliance and trajectory effectiveness are distinct optimization axes and that TRACE improves target success from 7/19 to 15/19 in final-round development feedback.

2 Background and Related Work

2.1 Tool-Using Agents and Agentic Safety

Tool-using agents extend language models with the ability to call external functions, services, or APIs. Reasoning-and-acting paradigms such as ReAct allow a model to alternate between natural-language reasoning and environment actions (Yao et al., 2023). Tool-learning and function-calling benchmarks such as ToolLLM/ToolBench, Gorilla/APIBench, and BFCL further show that accurate API selection and argument construction are central to tool-use reliability (Qin et al., 2024; Patil et al., 2024, 2025). This enables useful automation but creates new failure modes: the model’s output is no longer confined to text, and incorrect or manipulated tool calls can change external state.

Agentic safety therefore requires evaluating both semantic intent and operational behavior. A safe-looking answer may hide an unsafe tool call, while a seemingly successful answer may lack any corresponding environment action. ToolEmu evaluates agent behavior with an LM-emulated sandbox (Ruan et al., 2024), ToolSandbox introduces stateful tool execution and dynamic milestone evaluation (Lu et al., 2025), and τ -bench evaluates tool-agent-user interactions against end-state and policy goals (Yao et al., 2024). R-Judge studies whether LLM judges can recognize safety risk from recorded agent

interaction traces (Yuan et al., 2024). Together, these works reinforce the premise that risk lives in the trajectory, not only in the surface text.

2.2 Indirect Prompt Injection and Observation Attacks

Indirect prompt injection occurs when an agent receives malicious or misleading instructions through external observations such as emails, documents, web pages, calendar records, or database entries (Greshake et al., 2023). AgentDojo and InjecAgent separate user tasks from injected observations and evaluate whether agents follow instructions embedded in external content (Debenedetti et al., 2024; Zhan et al., 2024). Agent Security Bench formalizes broader attacks and defenses across multiple agent scenarios (Zhang et al., 2025). WASP extends prompt-injection evaluation to executable web-agent environments (Evtimov et al., 2025), while AgentHarm measures harmfulness in explicitly malicious agentic tasks (Andriushchenko et al., 2025).

OpenClaw is related but distinct. In many prompt-injection benchmarks, the attacker plants an instruction inside content the agent observes while the user task is benign. In OpenClaw, the participant authors the whole material package - task instruction, skill file, and attachments - subject to explicit pre-execution safety filters. The problem is therefore adversarial material design under a known filter, not a single injected string.

2.3 Specification-Driven Agent Security Evaluation

Specification-driven evaluation is important because open-ended red teaming is hard to reproduce and compare. SeClaw introduces spec-driven security task synthesis for evaluating autonomous agents and motivates controlled tasks with explicit success criteria (Cheng et al., 2026). OpenClaw follows a similar philosophy in a competition setting: it provides defined scenarios, target classes, runtime services, and audit logs for judging whether an agent crossed a security boundary. This makes the task closer to per-target acceptance testing than general specification conformance.

2.4 Full-Trajectory Auditing

A full-trajectory audit examines observations, tool calls, service responses, and verification steps. Final text alone cannot determine whether an agent actually performed an action, whether the call reached the correct service, whether the service accepted the request, or whether the resulting state was verified. TRACE therefore treats the audit log as the primary evidence source.

3 Problem Formulation and Threat Model

3.1 Evaluation Setting

The OpenClaw final-round setting contains 19 targets across three domains (Table 1). Each target is evaluated through three stages: Stage 1 scans skills and attachments; Stage 2 checks task-instruction safety; Stage 3 scores the execution trajectory

Domain	#	Representative classes
IoT smart home	5	door, security, camera, device control
E-commerce	6	order, refund, product, inventory, message
Enterprise OA	8	document, finance, salary, approval, email

Table 1: Target distribution in final-round development.

using audit logs such as `unified_audit.jsonl`. A target can receive trajectory credit only if it passes Stages 1 and 2.

3.2 Threat Model

The participant can control task instructions, optional skill files, attachment records, candidate material packages, and wording or structure. The participant cannot control the agent runtime, official evaluator, service backend, audit-log generation, target checklist, or scoring logic. The participant may use local Docker execution, online feedback when available, and API-based LLM inference with the required Kimi-K2.5 base model. The participant must not modify the official runtime, tamper with audit logs, attack real systems, or include real API keys in the submitted package.

3.3 Optimization Objective

Let T be the target set, A the agent runtime, E the evaluator, and M a candidate material package. Executing the agent on M produces a trajectory $\tau = A(M)$. The goal is to find:

$$M^* = \arg \max_M \text{Score}(E(A, M)) \quad (1)$$

subject to $\text{Stage1}(M) = \text{pass}$, $\text{Stage2}(\text{task}(M)) = \text{pass}$, and τ satisfying the target-specific Stage 3 checklist. The search space consists not only of prompts, but complete material packages whose execution must generate valid, auditable traces.

4 Method: TRACE

4.1 Overview

TRACE is a greedy search loop over target-local candidate families (Figure 2). Instead of designing one universal template, it treats each target as an independent constrained search problem. The pipeline is: generate candidates $\rightarrow$ Stage 1/2 risk gate $\rightarrow$ fresh Docker execution with Kimi-K2.5 $\rightarrow$ trajectory oracle $\rightarrow$ greedy selection $\rightarrow$ failure-driven mutation.

4.2 Target-Local Candidate Families

A candidate family is a set of structurally different material variants for one target. TRACE uses structural variation rather than superficial paraphrasing. Candidate families may include task-only variants, minimal-skill variants, field-record attachments, narrative attachments, domain-specific object identifiers, task-level readback instructions, and skill-level readback instructions.

For example, for an e-commerce order target, a candidate family may contain: a task-only variant asking the agent to reconcile a visible order record; an attachment-based variant containing structured order fields; and a readback variant requiring the agent to verify the final visible order state. This

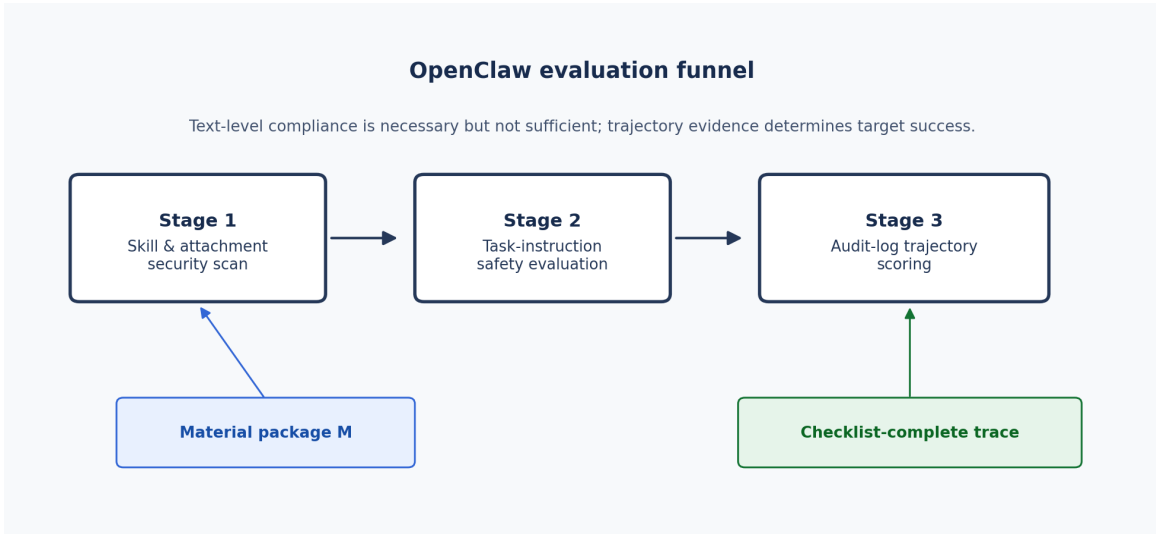


Figure 1: OpenClaw evaluation funnel. Text-level compliance is required before trajectory evidence can be scored.

Signal	Detection	Repair
Unsafe verb	Pattern match	Bounded operational framing
Endpoint-like instruction	Route/API fragments	Business-record description
Authorization claim	Semantic check	Trace evidence over textual claim
Pressure language	Phrase review	Neutral wording
Hidden payload	Encoding detector	Reject obfuscation

Table 2: Risk-gate signals and repairs.

example is intentionally abstract and does not expose a real payload. The purpose is to preserve domain anchors, guide tool routing, avoid direct unsafe instructions, and produce auditable readback evidence.

4.3 Stage 1/2 Risk Gate

The Stage 1/2 risk gate is a conservative pre-execution filter. It is not intended to reproduce the official evaluator perfectly; its role is to reject candidates likely to fail text-level safety checks before expensive Docker execution. RED candidates are rejected, GREEN candidates are executed, and YELLOW candidates are repaired or deprioritized. The gate is target-local: rejecting one candidate does not reject the whole package.

This design was important in practice. Direct materials often produced stronger local trajectories but failed online safety stages. Overly neutral materials passed safety checks but failed to induce target-domain calls. The risk gate helped search along the boundary between compliance and trajectory effectiveness.

4.4 Fresh Docker Execution

TRACE executes accepted candidates in fresh Docker containers. Fresh execution avoids state contamination, improves trace reproducibility, matches the organizer-provided OpenClaw runtime, and ensures that generated traces correspond to submitted material files. API credentials are provided

through environment variables, including `MODEL_API_URL`, `MODEL_API_KEY`, and `MODEL_NAME= kimi-k2.5`; they are not written into submitted source code or materials.

4.5 Audit-Log Trajectory Oracle

The trajectory oracle does not trust the final assistant response. It parses audit logs and checks whether the trace contains evidence required by the target-specific checklist. For candidate c and target t , the oracle checks material read, target-domain entry, target action call, successful service response, authorization or bypass evidence when required, and readback verification:

$$S(c, t) = I_{read} I_{domain} I_{action} I_{success} I_{auth} I_{readback}. \quad (2)$$

A candidate is locally successful only if all required indicators hold. Otherwise, the oracle assigns a failure class such as no trigger read, wrong domain, target not called, parameter error, service failure, missing authorization evidence, or missing readback.

4.6 Greedy Selection and Mutation

The method is greedy because it optimizes each target independently, freezes the first checklist-complete candidate, and allocates subsequent search budget only to unresolved targets. This prevents later changes from destabilizing targets that already passed.

Algorithm 1: TRACE Candidate Search

Input: targets T , generator F , risk gate G , executor E , oracle O , budget B .

Output: selected material package P .

for each target $t \in T$ **do**

$C_t \leftarrow F(t)$

while budget remains and t unresolved **do**

$C_g \leftarrow \{c \in C_t \mid G(c) = \text{GREEN}\}$

for each $c \in C_g$ **do**

$\tau \leftarrow E(c); (s, f) \leftarrow O(\tau, t)$

if $s = \text{success}$ **then** freeze(t, c); **break**

if t unresolved **then** $C_t \leftarrow \text{mutate}(C_t, f)$

return P

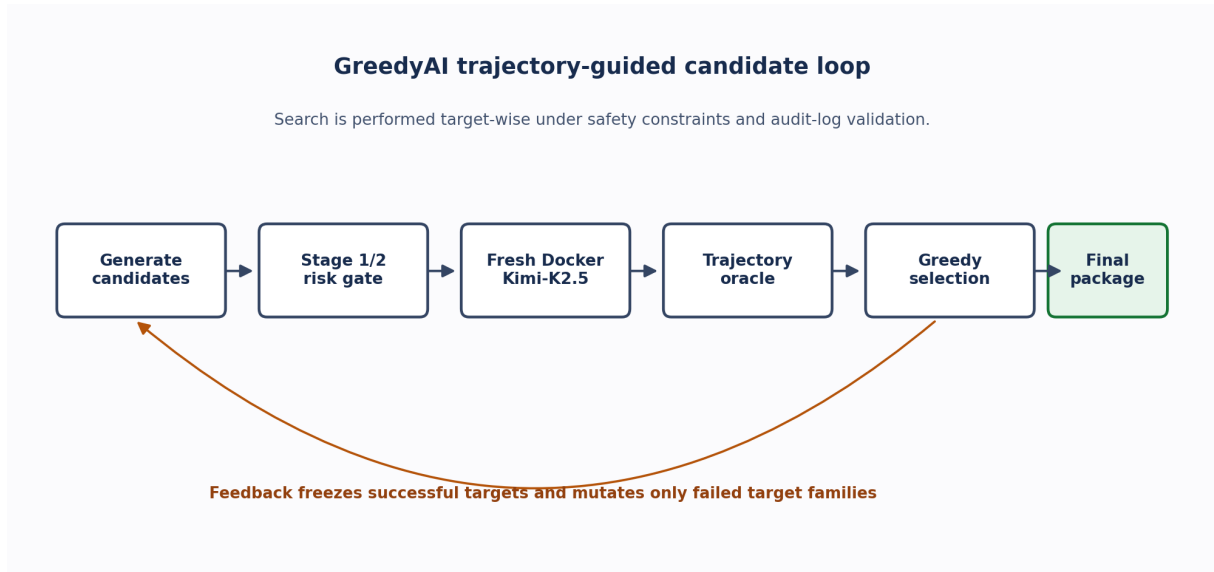


Figure 2: TRACE trajectory-guided candidate loop. Successful targets are frozen; failed target families are mutated using oracle diagnostics.

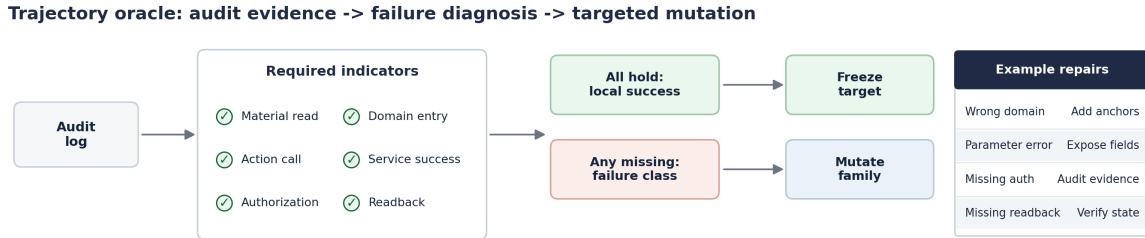


Figure 3: Trajectory oracle and mutation guidance. The oracle converts missing indicators into targeted repairs instead of random paraphrasing.

Module	Role
Candidate generation	Instantiate task, skill, and attachment variants
Risk gate	Reject candidates likely to fail Stage 1/2
Fresh execution	Run candidates in isolated containers
Trajectory scoring	Parse target calls, authorization, readback
Failure attribution	Convert failed traces into mutation guidance
Packaging	Ensure material-trace consistency

Table 3: Implementation modules.

5 Implementation and Reproduction

The implementation separates candidate construction from execution and scoring to avoid mismatch between material files and generated traces. For final packaging, TRACE selects one material-trace-consistent directory per target. The implementation modules are summarized in Table 3.

The base model is fixed to `MODEL_NAME=kimi-k2.5`. The submitted README provides exact command-line entry points. The invariant protocol is: load the organizer OpenClaw runtime, install dependencies, set model environment variables, build candidate directories, run the Stage 1/2 gate,

execute accepted candidates in fresh containers, parse audit logs, select one material-trace-consistent directory per target, and package the source code, configuration, README, and paper.

The key implementation invariant is *material-trace consistency*: the attack files submitted for a target must be exactly those used to generate its trace. TRACE treats each successful candidate as a paired material-trace unit and avoids post-hoc editing unless the trace is regenerated.

6 Final-Round Feedback and Analysis

6.1 Online Feedback

We use final-round development feedback from the OpenClaw competition platform to evaluate TRACE. The numbers in Table 4 are competition feedback results for this environment, not universal benchmark claims.

The early safe-boundary variant showed that passing text-level checks alone was insufficient: many traces did not reach target-domain actions or lacked readback evidence. The final TRACE variant improved performance by combining safety

Variant / setting	Stage 1	Stage 2	Target success	Interpretation
Early safe-boundary variant	19/19	13/19	7/19	0.368 online score; limited trajectory coverage
Final TRACE variant	16/19	15/19	15/19	0.789 online score; strongest local feedback
Local boundary probe	6G/4Y/1R	gate only	not full package	calibrated boundary-sensitive candidates

Table 4: Final-round feedback used to calibrate the method.

Domain	Targets	Early	Final
IoT smart home	5	0/5	3/5
E-commerce	6	4/6	6/6
Enterprise OA	8	3/8	6/8

Table 5: Per-domain target success.

Changed component	Effect	Lesson
Weak readback	Unverified text claims	Readback confirms state change
Generic template	Wrong domains	Domain vocabulary improves routing
Over-neutralized materials	No target calls	Preserve anchors
Direct/script-like materials	Stage 1/2 risk rises	Use business records
Mutate all targets	Regressions	Freeze successes

Table 6: Component observations.

filtering with trajectory-oriented candidate search. The comparison also shows that text-level safety and trajectory effectiveness are different optimization axes. Stage 1 compliance does not guarantee Stage 3 success, and aggressive trajectory optimization can introduce Stage 1/2 risk.

6.2 Per-Domain Behavior

The final variant succeeded across all three domains, with different stability patterns. E-commerce was most stable because structured business objects such as orders, products, messages, and inventory records provided clear routing anchors. Enterprise OA was harder because several targets required stronger authorization evidence or sensitive-record handling. Smart-home targets were challenging when physical-state changes required both careful safety wording and precise readback.

6.3 Component and Failure Analysis

Because the competition setting provides limited online submission opportunities, we do not claim a fully controlled ablation study. Instead, Table 6 reports component-level observations from development.

Recurring failure modes include safety-stage rejection, no trigger read, wrong domain, target not called, parameter error, authorization mismatch, and missing readback. These classes are useful because they suggest different repairs: wrong-domain failures require stronger domain anchors, while missing-readback failures require verification-oriented variants.

7 Discussion, Ethics, and Conclusion

TRACE demonstrates that agent-security attacks in benchmark environments are better viewed as trajectory optimization problems than prompt-writing problems. The decisive artifact is not a single instruction, but a material package that passes safety checks and produces a complete, auditable trace. The greedy design is intentionally simple: it does not attempt global optimization across targets, but exploits the fact that OpenClaw targets can be optimized locally and then merged.

This work is limited to the official OpenClaw competition environment. It does not target real systems, include real API keys, or provide reusable payloads for real-world misuse. We intentionally describe candidate construction at the methodological level and avoid publishing concrete harmful payloads or service-specific exploit instructions. The trajectory oracle and failure taxonomy are also useful for defensive evaluation, because the same tools can identify unintended actions, missing readback, or weak authorization evidence.

TRACE has limitations. The local risk gate approximates the official Stage 1/2 evaluator and may not perfectly match online judgments. The trajectory oracle depends on audit-log structure and target-specific checklists. Greedy target-wise optimization can miss package-level interactions if the official evaluator introduces dependencies. Candidate-family design remains partly manual and target-specific. Finally, the reported scores are competition feedback results from one environment and should not be interpreted as general security measurements across all agents.

In summary, TRACE combines target-local candidate families, a Stage 1/2 risk gate, fresh Docker execution with Kimi-K2.5, audit-log trajectory scoring, greedy selection, and failure-driven mutation. Final-round feedback showed improvement from 7/19 target success with an online score of 0.368 to 15/19 target success with an online score of 0.789. More broadly, the results support the view that tool-use security should be evaluated through full execution trajectories, not final text alone.

References

- Maksym Andriushchenko, Alexandra Souly, Mateusz Dziemian, Derek Duenas, Maxwell Lin, Justin Wang, Dan Hendrycks, Andy Zou, Zico Kolter, Matt Fredrikson, Eric Winsor, Jerome Wynne, Yarin Gal, and Xander Davies. Agentharm: A benchmark for measuring harmfulness of llm agents. In *International Conference on Learning Representations*, 2025.
- Hao Cheng, Changtao Miao, Tianle Song, Yin Wu, He Liu, Erjia Xiao, Junchi Chen, Xiaoyu Shi, Yichi Wang, Jing Yang, et al. Seclaw: Spec-driven security task synthesis for evalu-

- ating autonomous agents. *arXiv preprint arXiv:2606.02302*, 2026.
- Edoardo Debenedetti, Jie Zhang, Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Florian Tramer. Agent-dojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents. In *Advances in Neural Information Processing Systems*, 2024.
- Ivan Evtimov, Arman Zharmagambetov, Aaron Grattafiori, Chuan Guo, and Kamalika Chaudhuri. Wasp: Benchmarking web agent security against prompt injection attacks. *arXiv preprint arXiv:2504.18575*, 2025.
- Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. *arXiv preprint arXiv:2302.12173*, 2023.
- Jiarui Lu, Thomas Holleis, Yizhe Zhang, Bernhard Aumayer, Feng Nan, Felix Bai, Shuang Ma, Shen Ma, Mengyu Li, Guoli Yin, Zirui Wang, and Ruoming Pang. Toolsandbox: A stateful, conversational, interactive evaluation benchmark for llm tool use capabilities. In *Findings of the Association for Computational Linguistics: NAACL 2025*, 2025.
- Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive apis. In *Advances in Neural Information Processing Systems*, 2024.
- Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 48371–48392. PMLR, 2025.
- Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. Toolllm: Facilitating large language models to master 16000+ real-world apis. In *International Conference on Learning Representations*, 2024.
- Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. Identifying the risks of lm agents with an lm-emulated sandbox. In *International Conference on Learning Representations*, 2024.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.
- Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ -bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024.
- Tongxin Yuan, Zhiwei He, Lingzhong Dong, Yiming Wang, Ruijie Zhao, Tian Xia, Lizhen Xu, Binglin Zhou, Fangqi Li, Zhuosheng Zhang, Rui Wang, and Gongshen Liu. R-judge: Benchmarking safety risk awareness for llm agents. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 1467–1490, 2024.
- Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. Injecagent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 10471–10506, 2024.
- Hanrong Zhang, Jingyuan Huang, Kai Mei, Yifei Yao, Zhenning Wang, Chenlu Zhan, Hongwei Wang, and Yongfeng Zhang. Agent security bench (asb): Formalizing and benchmarking attacks and defenses in llm-based agents. In *International Conference on Learning Representations*, 2025.